

6. Coding and Testing

(5 hours, 7 marks)

By Bidur Sapkota

6.1 Coding Standards and Guidelines

Coding standards are a set of rules and conventions that a development team agrees to follow when writing source code. They govern how code is structured, named, formatted, and documented. The purpose is to produce code that is consistent, readable, maintainable, and less error-prone across the entire team.

Importance of coding standards:

- **Readability**: Consistent code is easier for any team member to read and understand, not just the original author.
- **Maintainability**: Since 60–80% of total software effort is spent on maintenance, code that follows uniform conventions is significantly cheaper to modify and extend.

6.1 Coding Standards and Guidelines

Importance of coding standards:

- **Reduced errors**: Standards enforce practices (e.g., proper initialization, boundary checking) that prevent common coding mistakes.
- **Easier code reviews**: Reviewers can focus on logic and correctness rather than style inconsistencies.
- **Onboarding**: New team members can understand the codebase faster when it follows predictable conventions.

6.1 Coding Standards and Guidelines

Areas of coding standards:

Naming conventions: Variable, function, class, and constant names should be meaningful and follow a consistent scheme. Common conventions include:

camelCase for variables and functions (e.g., `totalPrice` , `calculateTax()`),

PascalCase for classes (e.g., `OrderManager` , `CustomerAccount`), and

UPPER_SNAKE_CASE for constants (e.g., `MAX_RETRIES` , `DEFAULT_TIMEOUT`).

Names should be descriptive enough to convey purpose without requiring additional comments.

6.1 Coding Standards and Guidelines

Areas of coding standards:

Indentation and formatting: A consistent number of spaces (typically 2 or 4) or tabs should be used for indentation. Line length should be limited (commonly 80–120 characters). Braces, parentheses, and whitespace around operators should follow a single, agreed-upon style. Consistent formatting makes the control structure of code visually apparent.

6.1 Coding Standards and Guidelines

Areas of coding standards:

Commenting and documentation: Comments should explain why something is done, not what is done (the code itself should be clear enough for the "what").

Every module or file should have a header comment describing its purpose, author, and date. Complex algorithms should be annotated. Over-commenting (restating obvious code) should be avoided as it clutters the codebase.

Code organization: Related functions should be grouped logically. Files and directories should follow a consistent project structure. Dead code (unused functions, commented-out blocks) should be removed.

6.1 Coding Standards and Guidelines

Areas of coding standards:

Error handling: All functions should handle error conditions explicitly. Return values should be checked. Exception handling should follow a defined pattern across the project.

Coding guidelines vs. coding standards: Standards are strict rules that must be followed (e.g., "all class names must use PascalCase"). Guidelines are recommended best practices that allow some flexibility (e.g., "prefer composition over inheritance where possible"). Both contribute to code quality, but standards are mandatory and enforceable, while guidelines are advisory.

6.2 Code Review

What is a software inspection process? [4 marks] (2082 Bhadra - IOE - Old Syllabus Relevant)

What are the faults that can be uncovered by software inspection? [2 marks] (2082 Baishakh - IOE - Old Syllabus Relevant)

Why are program inspections an effective technique for discovering errors? [5 marks] (2068 Chaitra - IOE - Old Syllabus Relevant)

What types of errors are unlikely to be discovered through inspections? [5 marks] (2068 Chaitra - IOE - Old Syllabus Relevant)

6.2 Code Review

Code review is the systematic examination of source code by peers to find errors, improve quality, and share knowledge. It is a form of static testing because code is analyzed without being executed. Reviews are one of the most effective techniques for early defect detection. Industry studies show that review techniques can be up to 75% effective in uncovering design flaws.

Importance of Code review:

- An error found during review costs far less to fix than one found during testing or after deployment. A requirements error found during review may cost about 6 person-hours to fix, while the same error found during testing costs an average of 45 person-hours.

6.2 Code Review

Importance of Code review:

- Reviews catch defects that testing often misses, such as logic errors, poor algorithmic efficiency, security vulnerabilities, and violations of standards.
- Reviews don't just find bugs; they also improve team knowledge, enforce coding standards, and promote consistency across the codebase.

Defect amplification: An error introduced early in the software process (e.g., during requirements) and left undetected can amplify into multiple errors during design, and those can further amplify during coding. A single uncorrected requirements error can cascade into dozens of coding errors. Reviews at each stage break this amplification chain.

6.2.1 Code Walkthrough

Code walkthrough is an informal, author-led peer review in which the developer guides a small group of colleagues through the code, explaining the logic and design decisions.

Process:

- The author organizes the session and presents the code.
- Participants (typically 2–5 peers) listen, ask questions, and offer suggestions.
- The author "walks" the reviewers through the code line-by-line or function-by-function.
- Discussion is relatively free-form and collaborative.
- Notes may be taken informally, but there is no formal follow-up process.

6.2.1 Code Walkthrough

Characteristics:

- Low formality: minimal advance preparation is required from reviewers.
- Author-driven: the author controls the pace and focus of the session.
- Best used for knowledge sharing, brainstorming, and getting quick feedback.
- Effective at catching high-level design problems and sharing context among team members.
- Less effective at finding subtle defects compared to formal inspections, because reviewers have not studied the code in advance.

6.2.2 Code Inspection

Code inspection (also called a Fagan inspection, after Michael Fagan who formalized the process at IBM in 1976) is a highly formal, structured peer review process specifically designed for systematic defect detection.

Process (distinct phases):

- 1. Planning:** The inspection leader selects the review team (typically 3–5 people), distributes materials, and sets the schedule.
- 2. Overview (optional):** The author provides background on the work product for the reviewers.
- 3. Preparation:** Each reviewer independently examines the code using predefined checklists, noting potential defects. This typically requires 1–2 hours per reviewer.

6.2.2 Code Inspection

Process (distinct phases):

- 4. Inspection meeting:** A trained moderator (not the author) leads the meeting. A designated reader (not the author) walks through the code. A recorder logs every defect found. The author answers questions but does not lead. The meeting is time-limited (typically under 2 hours).
- 5. Rework:** The author corrects all identified defects.
- 6. Follow-up:** The moderator verifies that all defects have been properly corrected.

6.2.2 Code Inspection

Characteristics:

- High formality: defined roles (moderator, reader, recorder, author), checklists, metrics.
- Not author-driven: the moderator controls the process, and a reader (not the author) presents the code.
- Focused on defect detection, not problem solving. Solutions are not discussed during the inspection meeting.
- Produces formal records: an issues list and a summary report.

6.2.2 Code Inspection

Characteristics:

- The review team decides whether to:
 - (1) accept the work product without modification,
 - (2) accept provisionally with minor corrections (no re-inspection needed), or
 - (3) reject due to severe errors (re-inspection required after corrections).

6.2.2 Code Inspection

Faults uncovered by inspection: Logic errors, incorrect or missing functionality, violations of coding standards, data type mismatches, incorrect variable initialization, boundary condition errors, unreachable code, incorrect interface usage, and security vulnerabilities.

Errors unlikely to be discovered through inspection: Performance bottlenecks under real load, timing and concurrency issues, hardware-dependent failures, usability problems, and errors that only manifest through complex multi-module interactions during execution.

6.2.2 Code Inspection

Formal Technical Reviews (FTR) guidelines:

- Review the product, not the producer. The tone should be constructive, not adversarial.
- Set an agenda and maintain it. Avoid drift.
- Limit debate and rebuttal. Record issues for offline discussion.
- Enunciate problem areas, but don't attempt to solve problems during the meeting.
- Limit the number of participants and insist on advance preparation.
- Develop checklists for each type of work product.
- Allocate dedicated resources and schedule time for reviews.

6.2.3 Cleanroom Technique

Cleanroom software engineering technique is a rigorous development methodology that emphasizes defect prevention rather than defect removal. It was developed by Harlan Mills at IBM and draws its name from hardware cleanroom manufacturing, where contamination is prevented rather than cleaned up afterward.

Principles:

- **No unit testing by developers.** Developers never compile or execute their own code for debugging purposes. The rationale is that reliance on testing encourages sloppy coding; removing the "safety net" of debugging forces developers to think more carefully.

6.2.3 Cleanroom Technique

Principles:

- **Formal verification**. Instead of testing, developers use mathematical correctness verification (also called functional verification) to demonstrate that the code is consistent with its specification. Each code segment is verified by the developer through logical reasoning.
- **Incremental development**. Software is developed in small increments. Each increment is formally specified, designed, verified, and then passed directly to an independent testing team.

6.2.3 Cleanroom Technique

Principles:

- **Statistical usage testing.** An independent testing team (separate from developers) tests each increment using statistical usage testing, where test cases are generated based on the expected usage profile of the software. This tests the software the way real users would use it, giving a statistically valid measure of reliability.
- **Statistical quality certification.** The results of usage testing are fed into reliability models to compute a mean time to failure (MTTF) for the software. If the computed reliability meets the specified target, the increment is certified. If not, it is returned to the developers for correction.

6.2.3 Cleanroom Technique

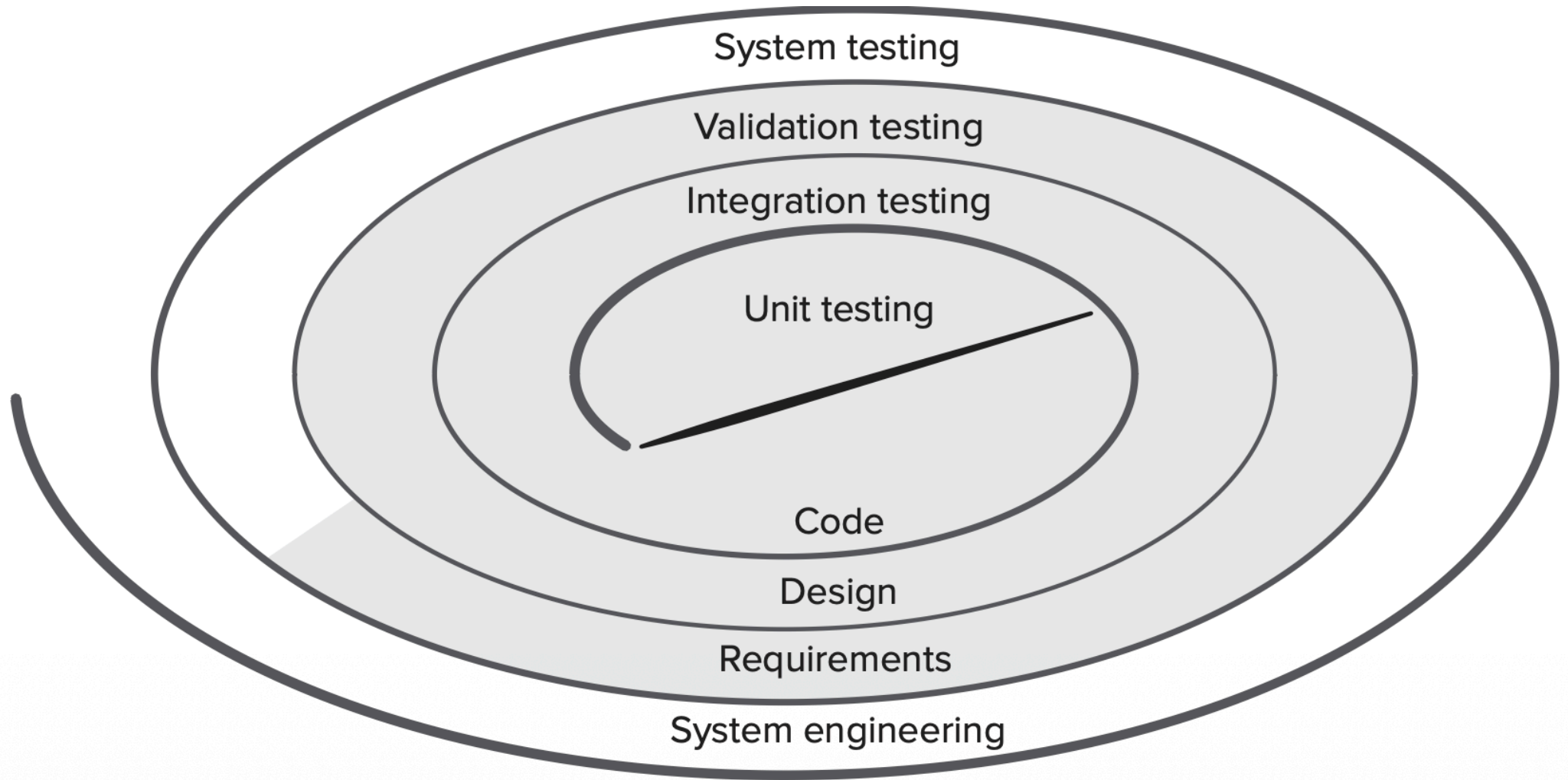
Traditional Approach	Cleanroom Approach
Development begins with coding, followed by testing, debugging, and fixing defects.	Development begins with formal specification, verification, and certification before release.
Testing is primarily used to find and remove defects.	Verification is used to prevent defects from being introduced.
Developers are responsible for testing their own code.	An independent certification team performs testing and evaluation.
Reliability is often unknown until the later stages of development.	Reliability is measured and certified for each software increment.

6.3 Software Testing Fundamentals

Software testing is the process of executing a program with the intent of finding errors. A good test is one that has a high probability of discovering an as-yet-undiscovered error. Testing cannot prove the absence of defects; it can only demonstrate their presence. Exhaustive testing (testing every possible input and path) is practically impossible for any non-trivial program.

6.3 Software Testing Fundamentals

**Testing
strategy**



6.3.1 Verification and Validation

Differentiate between verification and validation. [3 marks] (2082 Baishakh - IOE - Old Syllabus Relevant)

Why is verification and validation planning necessary? [2 marks] (2075 Chaitra - IOE - Old Syllabus Relevant)

Verification and validation (V&V) are two complementary activities that ensure software quality.

6.3.1 Verification and Validation

Verification: "Are we building the product right?" Verification checks that the software correctly implements a specific function or specification. It ensures that the output of each development phase conforms to the input specification for that phase. Verification activities include technical reviews, inspections, walkthroughs, and analysis of design models.

Validation: "Are we building the right product?" Validation checks that the software meets the customer's actual needs and requirements. It ensures the final product is traceable to customer expectations. Validation activities include acceptance testing, usability testing, and customer demonstrations.

6.3.1 Verification and Validation

Importance of V&V planning:

- It defines what will be tested, when, and by whom, preventing ad hoc and incomplete testing.
- It ensures that testing resources are allocated proportionally to risk.
- It establishes success criteria and exit criteria for each testing phase.
- It provides traceability from requirements through test cases, ensuring no requirement goes untested.

6.3.2 Unit Testing

Explain unit testing. [2 marks] (2078 Kartik - IOE - Old Syllabus Relevant)

Why is unit testing alone not enough for verifying a complex system? [4 marks] (2070 Ashad - IOE - Old Syllabus Relevant)

Unit testing focuses on the smallest testable unit of software, typically an individual function, method, or component. The goal is to verify that each unit performs its intended function correctly in isolation.

6.3.2 Unit Testing

What is tested during unit testing:

- **Interface:** Data flows correctly into and out of the unit (correct parameter types, order, and count).
- **Local data structures:** Variables maintain integrity throughout all steps of the algorithm.
- **Independent paths:** All execution paths through the control structure are exercised at least once.

6.3.2 Unit Testing

What is tested during unit testing:

- **Boundary conditions:** The unit operates correctly at the limits of input/output values (minimum, maximum, edge cases).
- **Error-handling paths:** Error conditions are handled properly and produce appropriate error messages.

6.3.2 Unit Testing

Scaffolding: Because a unit is not a standalone program, testing it requires supporting code:

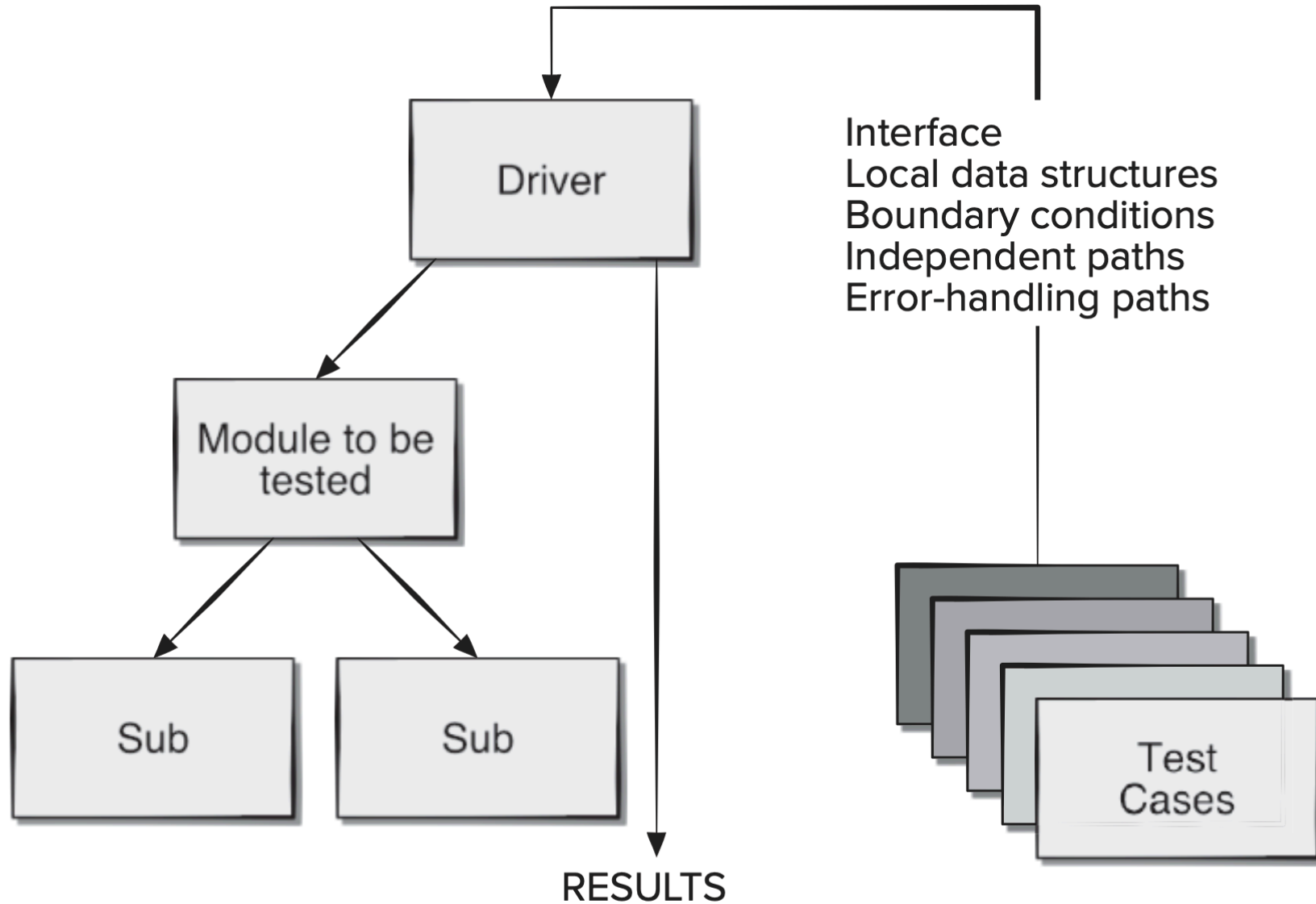
- A **driver** is a "main program" that calls the unit under test, passes it test data, and collects results. It simulates the part of the system that invokes the unit.
- A **stub** (or "dummy subprogram") replaces a module that the unit under test calls. It uses the correct interface, performs minimal processing, and returns control to the unit being tested.

6.3.2 Unit Testing

Why unit testing alone is not sufficient: Unit testing verifies individual components in isolation but cannot detect interface errors between components, integration problems, system-level performance issues, or defects that arise from the interaction of multiple units working together.

6.3.2 Unit Testing

**Unit-test
environment**



6.3.3 Integration Testing

Explain integration testing. [2 marks] (2078 Kartik - IOE - Old Syllabus Relevant)

Write about stub and driver testing. [3 marks] (2075 Chaitra - IOE - Old Syllabus Relevant)

Integration testing is the systematic process of combining unit-tested components and testing them as a group. The focus is on detecting errors in the interfaces and interactions between components.

6.3.3 Integration Testing

Importance:

Even if every individual unit works correctly in isolation, combining them can produce errors. Data may be lost across interfaces, one component may have unintended side effects on another, combined sub-functions may not produce the desired major function, and individually acceptable imprecisions may accumulate to unacceptable levels.

6.3.3 Integration Testing

Approaches to integration:

Big bang integration: All components are combined at once and the system is tested as a whole. This approach is discouraged because when errors are found, it is very difficult to isolate their cause within the vast expanse of the entire program.

Top-down integration: Integration starts with the main control module.

Subordinate modules are added one at a time (either depth-first or breadth-first).

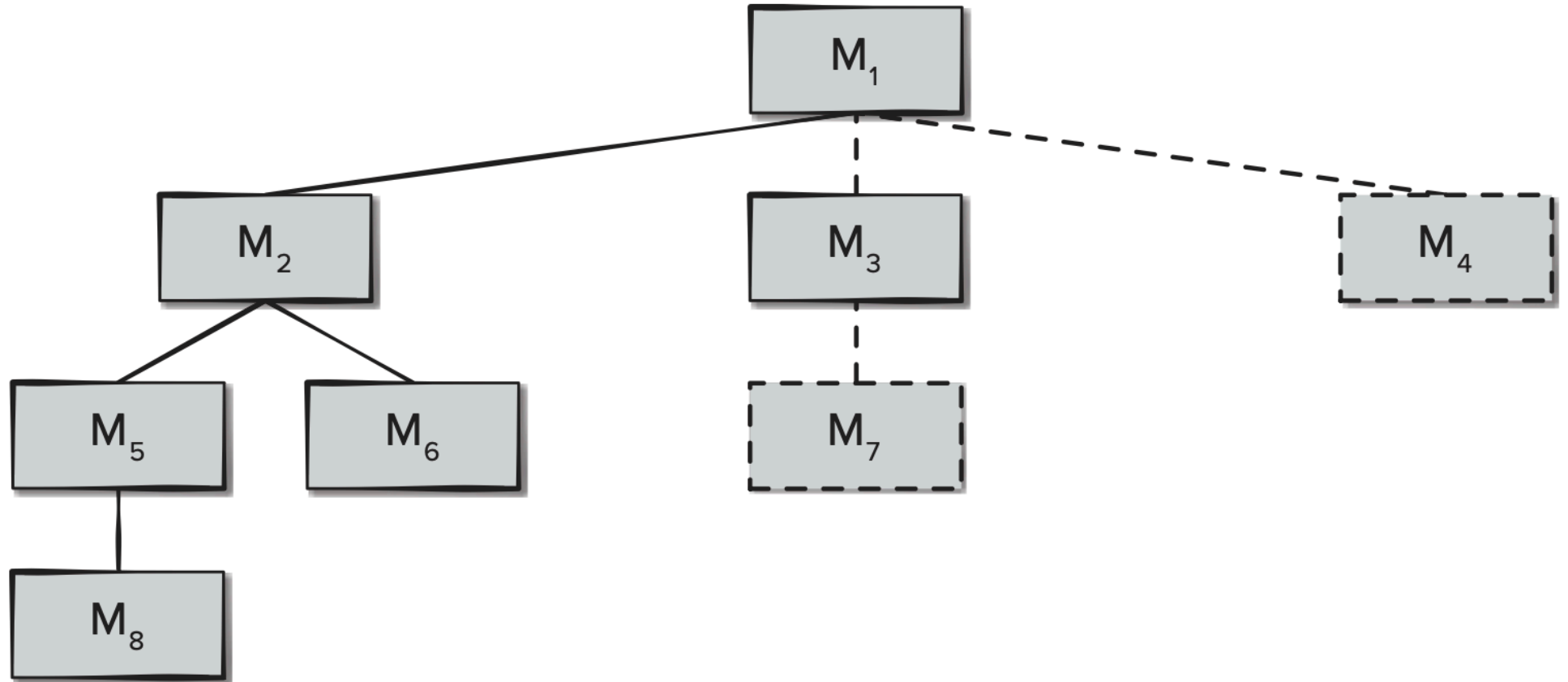
Stubs are used to replace modules that have not yet been integrated.

Advantage: major control and decision points are tested early.

Disadvantage: stubs may need to be complex.

6.3.3 Integration Testing

**Top-down
integration**



6.3.3 Integration Testing

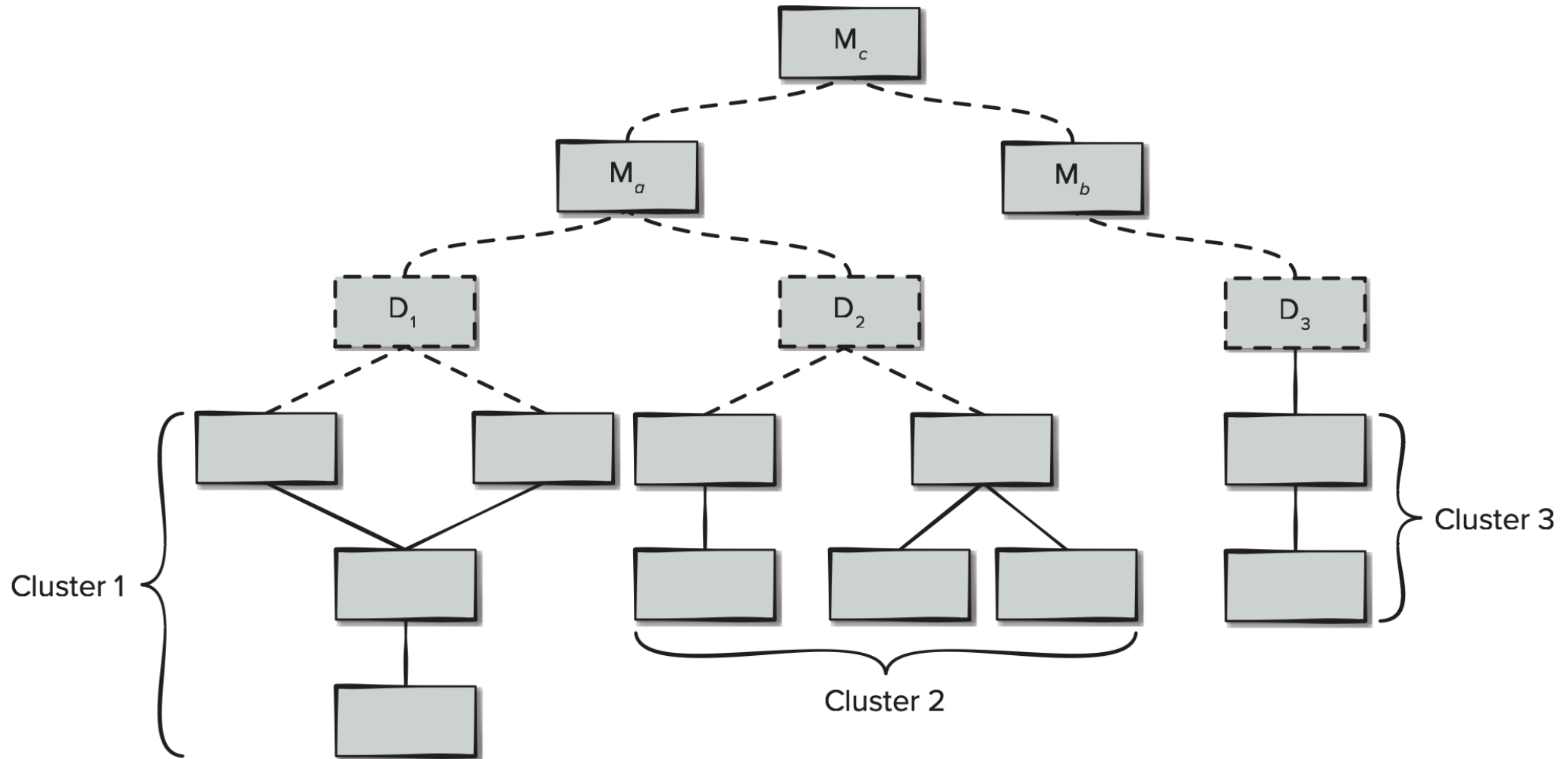
Approaches to integration:

Bottom-up integration: Integration begins with the lowest-level (atomic) modules, which are combined into clusters. Drivers are used to coordinate testing of each cluster. As integration progresses upward, drivers are removed and clusters are combined. Advantage: eliminates the need for complex stubs. Disadvantage: the complete system is not visible until the last module is added.

Continuous integration: Components are merged into the evolving software build one or more times per day. Each integration triggers automated tests. This is the standard practice in agile and DevOps environments. A variant called smoke testing involves rebuilding and testing the software daily to ensure the build is stable enough for further testing.

6.3.3 Integration Testing

FIGURE 20.2 Bottom-up integration



6.3.3 Integration Testing

Approaches to integration:

Regression testing: After each integration of a new component, a subset of previously passed tests is re-executed to ensure the new addition has not broken existing functionality. The regression test suite contains: (1) tests that exercise all software functions, (2) tests focusing on functions likely affected by the change, and (3) tests targeting the changed components.

6.3.3 Integration Testing

Stubs and drivers in integration testing:

Stubs are used in top-down integration to simulate lower-level modules not yet integrated. Drivers are used in bottom-up integration to simulate higher-level modules. Both represent testing overhead as they must be written but are not delivered with the final product.

6.3.4 System Testing

Explain system testing. [2 marks] (2078 Kartik - IOE - Old Syllabus Relevant)

Differentiate between system testing and integration testing. [2 marks] (2081 Baishakh - IOE - Old Syllabus Relevant)

System testing tests the fully integrated software system as a whole within the context of the broader computer-based system. It verifies that the software meets all functional, behavioral, and performance requirements when combined with other system elements such as hardware, databases, networks, and people.

6.3.4 System Testing

Types of system testing:

- **Recovery testing:** Forces the system to fail and verifies that recovery is properly performed (e.g., automatic restart, data integrity after crash).
- **Security testing:** Attempts to breach the system's protection mechanisms to verify that the system is resistant to unauthorized access.
- **Stress testing:** Pushes the system beyond its normal operating limits to determine how it degrades and at what point it fails.
- **Performance testing:** Tests run-time performance (response time, throughput, resource utilization) under expected and peak conditions.

6.3.4 System Testing

Difference between system testing and integration testing:

Integration testing focuses on verifying the interfaces and interactions between combined software components. System testing focuses on validating the entire system (software + hardware + environment) against the original system requirements and specifications.

6.3.5 Acceptance Testing

Differentiate between alpha testing and beta testing. [2 marks] (2081 Bhadra - IOE - Old Syllabus Relevant)

Acceptance testing is conducted by the end users (or customer) to determine whether the software meets their business requirements and is ready for deployment. It is the final testing phase before the software is released.

6.3.5 Acceptance Testing

Types of acceptance testing:

Alpha testing: Conducted at the developer's site by end users. The developer observes and records errors and usage problems. The environment is controlled by the developer.

Beta testing: Conducted at the end user's site in a real-world environment. The developer is generally not present. Users record problems and report them to the developer. Beta testing is a "live" test of the software by real users in their actual working conditions.

6.3.5 Acceptance Testing

Alpha Testing	Beta Testing
Alpha testing is conducted at the developer's site.	Beta testing is conducted at the user's site.
Alpha testing is performed by end users under the observation of developers.	Beta testing is performed by end users without direct observation from developers.
Alpha testing takes place in a controlled environment.	Beta testing takes place in a real world, uncontrolled environment.
Alpha testing aims to identify and fix errors before beta testing begins.	Beta testing aims to identify remaining issues before the final software release.

6.4 Black-Box and White-Box Testing Approach

Differentiate between black-box testing and white-box testing. [5 marks]

(2076 Chaitra - IOE - Old Syllabus Relevant)

Are both black-box and white-box testing necessary, or is just one sufficient? Justify. [3 marks] (2079 Bhadra - IOE - Old Syllabus Relevant)

What is equivalence partitioning? Explain with an example. [5 marks]

(2070 Ashad - IOE - Old Syllabus Relevant)

These are two fundamental and complementary philosophies for designing test cases. Both are necessary; neither alone is sufficient.

6.4 White-Box Testing

White-box testing (also called glass-box testing or structural testing) designs test cases based on knowledge of the internal structure, logic, and code of the software. The tester examines the source code and designs tests to exercise specific paths, conditions, and loops.

Ensure that all internal operations are performed according to specifications and that all internal code paths have been exercised.

6.4 White-Box Testing

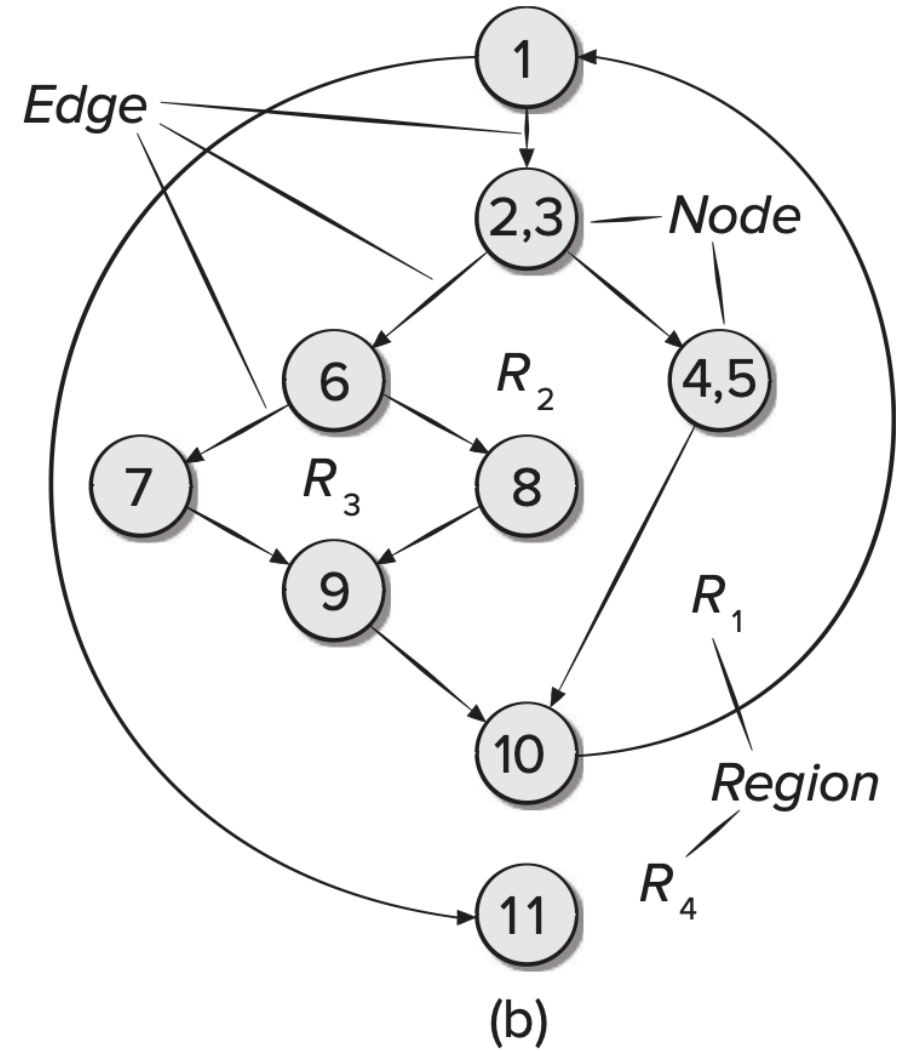
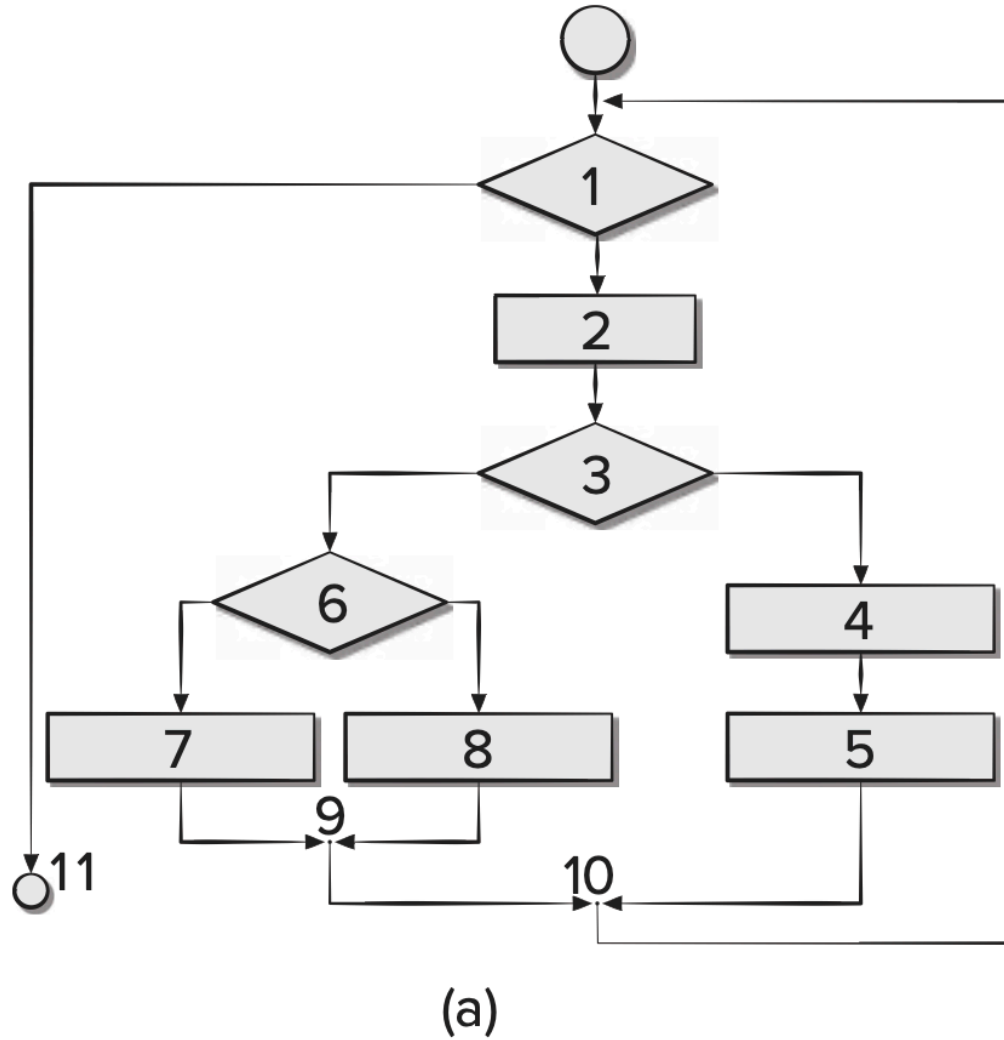
What white-box testing guarantees:

- All independent paths within a module are exercised at least once.
- All logical decisions are exercised on their true and false sides.
- All loops are executed at their boundaries and within their operational bounds.
- Internal data structures are exercised to ensure their validity.

A. Basis path testing: A technique proposed by Tom McCabe that uses cyclomatic complexity to determine the number of independent paths through a program. This defines the minimum number of test cases needed to ensure every statement is executed at least once.

6.4 White-Box Testing

(a) Flowchart
and (b) flow
graph



6.4 White-Box Testing

Cyclomatic complexity $V(G)$ is computed in three ways:

1. $V(G)$ = Number of regions in the flow graph.
2. $V(G) = E - N + 2$, where E = number of edges and N = number of nodes.
3. $V(G) = P + 1$, where P = number of predicate (decision) nodes.

The value of $V(G)$ gives an upper bound on the number of independent paths and, therefore, the minimum number of test cases needed for full statement coverage.

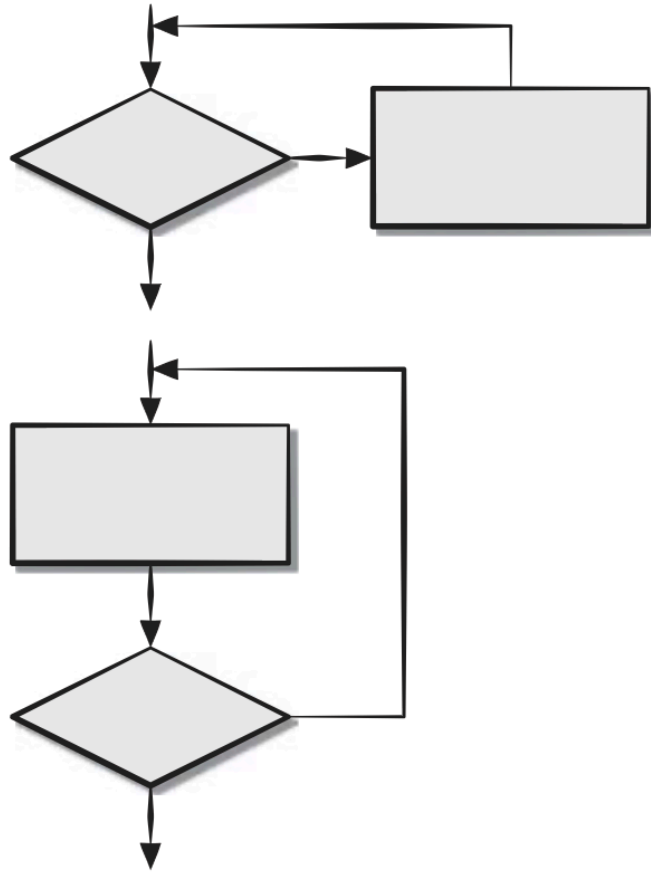
6.4 White-Box Testing

B. Control structure testing:

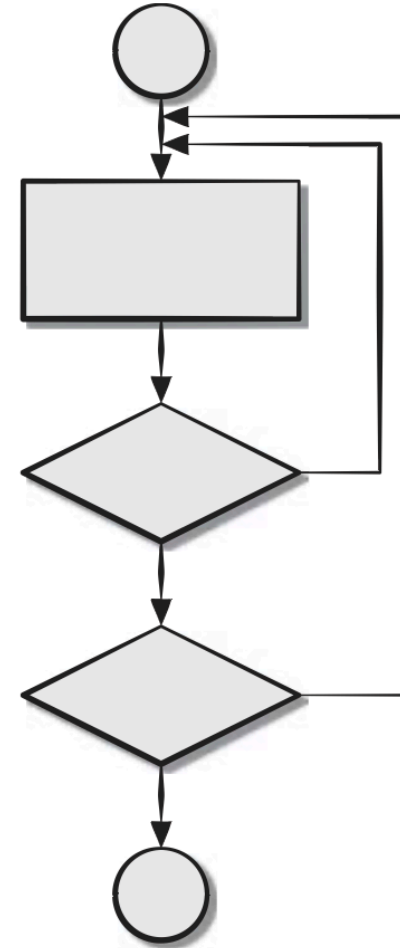
- **Condition testing:** Exercises all logical conditions in the program.
- **Data flow testing:** Selects test paths based on the locations of variable definitions and uses.
- **Loop testing:** Tests loop constructs specifically. For simple loops (with max n iterations): skip the loop entirely; 1 pass; 2 passes; m passes ($m < n$); and $n-1$, n , $n+1$ passes. For nested loops: start at the innermost loop, test it while holding outer loops at minimum values, then work outward.

6.4 White-Box Testing

Classes of loops



Simple loops



Nested loops

Black-Box Testing

Black-box testing (also called behavioral testing or functional testing) designs test cases based on the functional requirements and specifications of the software without knowledge of or regard for the internal code structure. The tester treats the software as a "black box" and focuses on inputs and expected outputs.

Black-Box Testing find errors in the following categories: (1) incorrect or missing functions, (2) interface errors, (3) errors in data structures or external database access, (4) behavior or performance errors, and (5) initialization and termination errors.

Black-Box Testing

A. Equivalence partitioning: Divides the input domain into classes (partitions) of data that are expected to be treated the same way by the software. One representative test case is selected from each class, reducing the number of test cases while maintaining coverage.

Rules for defining equivalence classes:

- If input specifies a range (e.g., 1–100): one valid class (values within range) and two invalid classes (below and above the range).
- If input specifies a specific value: one valid class and two invalid classes.

Black-Box Testing

Rules for defining equivalence classes:

- If input specifies a member of a set: one valid class (member of set) and one invalid class (not a member).
- If input is Boolean: one valid class (true) and one invalid class (false).

Example: An input field accepts ages 18–60. Equivalence classes: valid = {18–60}, invalid₁ = {< 18}, invalid₂ = {> 60}. Test cases: one from each class (e.g., 30, 10, 75).

Black-Box Testing

B. Boundary value analysis (BVA): A technique that focuses on testing at the edges of equivalence classes, because errors tend to occur at boundaries. For a range bounded by values a and b , test cases should include: a , b , $a-1$, $a+1$, $b-1$, $b+1$.

Example: For the age range 18–60, BVA test values would be: 17, 18, 19, 59, 60, 61.

White-Box & Black-Box Testing

White-Box Testing	Black-Box Testing
It is based on the internal structure and implementation of the code.	It is based on functional requirements and specifications.
It requires access to the source code.	It requires only the software specifications and expected behavior.
It focuses on logic paths, conditions, loops, and code coverage.	It focuses on input and output behavior without considering internal implementation.

White-Box & Black-Box Testing

White-Box Testing	Black-Box Testing
It is typically performed during the early stages of development, especially unit testing.	It is commonly performed during integration, system, and acceptance testing.
It helps identify logic errors, path errors, and data flow issues.	It helps identify missing functionality, interface defects, and behavioral errors.
It is also known as structural testing or glass-box testing.	It is also known as behavioral testing or functional testing.

White-Box & Black-Box Testing

Are both White-Box Testing and Black-Box Testing necessary?

Yes. White-box testing ensures internal correctness and coverage of all code paths. Black-box testing ensures the software meets functional requirements from the user's perspective. Each uncovers different classes of errors that the other is likely to miss. Using only one approach leaves significant categories of defects untested.

6.5 Agile Testing Practices

Agile methodologies integrate testing throughout the development lifecycle rather than treating it as a separate phase at the end. Testing is continuous, collaborative, and closely tied to user stories and acceptance criteria.

6.5.1 Test-Driven Development (TDD)

Test-Driven Development (TDD) is a software development practice in which automated unit tests are written before the production code. It follows a short, iterative cycle called Red-Green-Refactor:

6.5.1 Test-Driven Development (TDD)

- 1. Red - Write a failing test:** Before writing any production code, the developer writes a small, focused test case for the next piece of desired functionality. Running this test should fail (shown as "red" in testing tools) because the code being tested does not exist yet. This step confirms the test is valid and that the requirement is truly missing.
- 2. Green - Write the minimum code to pass:** The developer writes the simplest possible code that makes the failing test pass. The goal is correctness, not elegance. No optimization, no design patterns, just enough code to turn the test result from red to green.

6.5.1 Test-Driven Development (TDD)

3. Refactor - Improve the code: With the test passing as a safety net, the developer cleans up the implementation: removes duplication, improves names, simplifies logic, and improves structure. After refactoring, the test must still pass. The cycle then repeats for the next piece of functionality.

6.5.1 Test-Driven Development (TDD)

Benefits of TDD:

- Forces the developer to think about requirements and interface design before implementation.
- Produces a comprehensive suite of automated regression tests as a byproduct.
- Encourages small, modular, loosely coupled code.
- Provides immediate feedback on whether new code breaks existing functionality.
- Reduces debugging time because defects are caught at the moment the test fails.
- The test suite serves as living documentation of expected system behavior.

6.5.2 Continuous Testing

Continuous testing is the practice of executing automated tests as part of the software delivery pipeline, providing rapid feedback on code quality at every stage. It is a core component of CI/CD (Continuous Integration / Continuous Delivery) workflows.

How continuous testing works in a CI/CD pipeline:

1. A developer commits code to the shared repository.
2. The CI server (e.g., Jenkins, GitHub Actions, GitLab CI) automatically detects the change, builds the project, and runs the automated test suite.
3. If any test fails, the pipeline halts and the developer is notified immediately.
4. Only code that passes all tests is promoted to the next stage (integration testing, staging, deployment).

6.5.2 Continuous Testing

Characteristics:

- Tests run automatically on every code commit, not just at the end of a sprint or release cycle.
- Provides rapid feedback: developers learn about failures within minutes while the context is still fresh.
- Covers multiple levels: unit tests, integration tests, and sometimes acceptance tests are all automated.
- Enables the "shift left" approach, moving testing earlier in the development process to reduce the cost of defect correction.

6.5.3 Automated Unit Testing Basics

Automated unit testing uses testing frameworks to write, execute, and report on tests without manual intervention. Tests are written as code, stored alongside the production code, and executed automatically.

Common testing frameworks:

JUnit (Java), pytest (Python), NUnit (.NET), Jest (JavaScript) are widely used unit testing frameworks.

6.5.3 Automated Unit Testing Basics

Structure of an automated unit test (Arrange-Act-Assert):

- **Arrange:** Set up the test data, objects, and preconditions.
- **Act:** Execute the unit under test with the prepared inputs.
- **Assert:** Verify that the actual output matches the expected output.

6.5.3 Automated Unit Testing Basics

- Each test should be independent: it should not depend on the order of execution or the state left by other tests.
- Tests should be fast: a slow test suite discourages frequent execution.
- Tests should be deterministic: the same test should always produce the same result.
- Test names should clearly describe what is being tested and what the expected behavior is.
- Tests should cover both normal cases and edge/error cases.

6.5.4 Refactoring for Code Quality

Refactoring is the disciplined technique of restructuring existing code by altering its internal structure without changing its external behavior.

Purpose of refactoring:

- Improve code readability and understandability.
- Reduce complexity and eliminate duplication.
- Make the code easier to extend and maintain.
- Pay down technical debt (the accumulated cost of shortcuts and poor design decisions).

6.5.4 Refactoring for Code Quality

Code smells (indicators that refactoring is needed):

- **Long method:** A function that does too much and should be broken into smaller, focused methods.
- **Duplicate code:** The same or very similar code appears in multiple places.
- **Large class:** A class that has too many responsibilities and should be split.
- **Dead code:** Unused variables, functions, or code blocks that should be removed.
- **Feature envy:** A method that uses data from another class more than its own.
- **Magic numbers:** Hard-coded numeric values that should be replaced with named constants.

6.5.4 Refactoring for Code Quality

Common refactoring techniques:

- **Extract method:** Move a code fragment into a new method with a descriptive name.
- **Rename variable/method:** Replace unclear names with meaningful ones.
- **Replace magic number with named constant:** Replace `if (age > 18)` with `if (age > MIN_VOTING_AGE)`.
- **Inline method:** Replace a method call with the method's body if the method is trivially simple.

6.5.4 Refactoring for Code Quality

Common refactoring techniques:

- **Move method:** Relocate a method to the class where it logically belongs.
- **Decompose conditional:** Break a complex conditional expression into separate, well-named methods.

6.5.4 Refactoring for Code Quality

Refactoring and testing:

Refactoring must always be supported by a comprehensive suite of automated tests. Before refactoring, run all tests to confirm they pass. After each small refactoring step, re-run the tests to verify that behavior is preserved. This is why TDD and refactoring are closely linked: TDD produces the test suite that makes refactoring safe.